Towards Efficient Digital Twins: Task-specific vs Task-Agnostic Evaluation of Time-Series Lossy Compression

Emanuela Centra[1,2], Carlo Fabrizio[2], Gian Marco Paldino[2], and Gianluca Bontempi[2]

[1] Università di Parma, Parma, Italy
emanuela.centra@studenti.unipr.it
[2] Université libre de Bruxelles, Brussels, Belgium
{carlo.fabrizio,gian.marco.paldino,gianluca.bontempi}@ulb.be

Abstract. A *digital twin* (DT) is a virtual counterpart of a physical asset, continuously updated from its sensor streams. As more data is collected, it is typically stored in time-series databases (TSDBs). Storing this history at full resolution soon becomes untenable, so lossy compression is standard practice. Compression methods are usually chosen ignoring their effect on the downstream tasks that read the data. In the DT settings, data usually serves different tasks and, because the store is shared, a single compressed stream must serve the entire workload. Firstly, we demonstrate that the optimal compressor varies between datasets and tasks by using seven state-of-the-art compression methods, five multivariate datasets and a workload of three tasks. We then propose a framework for selecting the optimal *task-agnostic* compressor for such multi-task data. We motivate our work by quantifying the error incurred when a *task-specific* method is applied to the wrong task. The empirical analysis shows that, in the worst case scenario, the choice made using our proposed framework is on average much safer compared to task-specific one.

Keywords: time-series databases · lossy compression · multi-objective evaluation · digital twins · forecasting · classification · machine learning

1 Introduction

A digital twin (DT) is a virtual replica of a physical asset. It receives data from the asset's sensor stream and performs analysis that affect the physical counterpart [10].It retains the asset's history and might serve multiple analytical workloads from it. The "one stream, many tasks" pattern is the norm across deployed DTs [16]: heterogeneous workloads take place over the asset's life, from model training to drift detection, from what-if simulation to control.

Raw sensor history is usually retained in a time-series database (TSDB), a storage system specialised for the ingestion, retention, and query of time-stamped observations [28,14]. Storing multivariate data at native resolution can become

easily untenable. The remedy is *time-series compression*, either *lossless* (every original value is recoverable) or *lossy* (the recovered signal is not identical to the original one)[4]. Since a downstream task rarely needs a bit-exact reconstruction, lossy compression is the established practice in production TSDBs [35]. The choice of the compression method can be made offline, from a representative sample of the asset's stored history, or online, depending on the evolution of the data stream [30].

Whatever the moment of the choice, one method must serve every task the asset will run. Since different lossy compression methods preserve different characteristics of the signal, and different tasks exploit the data in different ways, the compressor that is best for one task may not be the best for another. Picking one compressor for a multi-task workload is therefore a trade-off.

To the best of the authors' knowledge, evaluating the commitment to a single compressor across different tasks and datasets has not been done yet. Our contribution is fourfold[3]:

- we propose a multi-objective evaluation criterion for lossy time-series compression methods;
- we define a metric, the *Price of Generality*, to measure the the excess loss incurred by committing to a single task-agnostic compressor instead of the best compressor chosen separately for each task;
- we empirically show that optimality of a compression method depends on both task and dataset;
- we propose a visual selection tool for practitioners.

The rest of the paper is structured as follows. Section 2 reviews the state-of-the-art (SOTA) on time-series compression. Section 3 presents our methodology: the problem formulation and the proposed approach. Section 4 describes the datasets, experimental settings, and evaluation protocol, then reports and discusses the results. Section 5 concludes and outlines directions for future work.

2 State of the Art

Current general-purpose lossy time series compression methods span rate-based decimation like the perceptual line simplification LTTB [25], block transforms based on spectrum properties (the discrete cosine [34] and wavelet [7] transforms), error-bounded piecewise-linear models (Sim-Piece [17]), channel projection (PCA [26]) and learned approaches, like autoencoders [3]. An important axis along which compression methods differ is how they treat the multivariate structure of the signal. *Channel-independent* (CI) families compress each channel on its own, while *channel-dependent* (CD) families compress the channels jointly, exploiting cross-channel correlation. Which is preferable is debated in

[3] All code and data are available at https://anonymous.4open.science/r/task-aware-lossy-compression-AI4DT-8B75/. The GitHub link will be provided upon acceptance.

the forecasting literature: Han et al. [11] find that the CD strategy has higher capacity but often lacks robustness under distribution drift and CI trades capacity for robust prediction. Zhao et al. [33] counter that: although CI methods are SOTA and reduce overfitting, they miss the accuracy available from channel dependence when it is modeled well, e.g. via leading indicators of the signal.

In general, works on time-series compression vary on two axes: proposing a new compressor or selecting one from a pool, and optimising for the signal or for the downstream task. Among the works proposing a new compressor focusing on a particular property of the signal, there is the autocorrelation-preserving CAMEO [21] and the seasonality-preserving framework of [19]. Both allow to achieve high compression ratios without compromising forecasting accuracy. A smaller line designs compression for the task. Landau and Nissim [18] propose adaptive downsampling that improves the generalisation of forecasting and classification models, and Miao et al. [20] a compression method that preserves monitoring quality in a digital twin while cutting storage. A third line does not introduce new methods, but selects one from a pool, optimised for the signal. This is typical in the TinyML/EdgeML setting, where choosing fast and keeping data small is crucial [5,27]. Model-based stores such as ModelarDB [15] fit an error-bounded model to each segment on ingest, keeping its parameters in place of the samples. Zhang et al. [32] online-select a lossless compressor from a pool of nine, each tailored to a stream type. In the fourth category, evaluating a pool of compressor against a specific downstream task, Muñiz-Cuza et al. [22] find forecasting degradation varies sharply between compressors (and at loose bounds compression can even denoise). Together these works do not establish that lossy compression is harmless. They show its effect ranges from degradation to improvement depending on the compressor, the usage, and the data, which is precisely why the choice cannot be made blind. All of these works measure the impact of a compressor on a single task and none give guidelines about which compressor to commit to when stored stream must serve several tasks at once.

Except for [29], the mentioned works treat selection as single-objective, but choosing a compressor is inherently a multi-objective one: a more faithful reconstruction generally costs more storage, so loss and storage compete and no compressor optimises both. This is the classic rate-distortion trade-off [6], and more generally a multi-objective optimisation problem [8]. A compressor is *Pareto-optimal* when no other option is better in all the aspects measured by the multi-objective criterion. The *Pareto front* of such compressors contains every defensible choice. Reducing the front to a single pick requires a preference. The standard device is a *scalarisation*, a weighted combination of the objectives, with the weight sweep recovering the front. Underwood et al. [29] stop after the construction of the Pareto front and do not describe how to commit to a single method. Moreover, they pair each dataset with a single task, without considering the possibility of using the same dataset for multiple tasks simultaneously. Our methodology addresses this issue by committing to a single compressor when the stored data must serve several tasks at the same time.

3 Methodology

Let $x \in \mathbb{R}^{r_x \times v_x}$ be a multivariate time-series dataset of r_x timesteps and v_x variables and $\mathcal{X}$ a pool of $n_x|$ datasets. A compressor c is a function that maps x to an encoded representation z (e.g., a compressed bitstream). A reconstructor d is a function that maps z back to a reconstructed signal $\hat{x}$, where $\hat{x}$ recovers the original dimensions ($r_{\hat{x}} = r_x$ and $v_{\hat{x}} = v_x$). A compressor c is *lossless* if there exists a reconstructor d such that $d(c(x)) = x$ for all x; otherwise, it is *lossy*. We define $\mathcal{C}$ as a set of $n_c = |\mathcal{C}|$ lossy compression algorithms.

We evaluate the downstream utility of the reconstructed signal using an *estimator* m. An estimator reads $\hat{x}$ and returns a scalar loss measuring the performance on a specific task. A workload comprises a set of tasks $t \in \mathcal{T}$. Each task may comprise several subtasks $u \in \mathcal{U}_t$. The estimators m of a task t form the set $\mathcal{M}_t$. The full set of estimators is $\mathcal{M} = \bigsqcup_{t\in\mathcal{T}} \mathcal{M}_t$, with total cardinality $n_m = |\mathcal{M}| = \sum_{t\in\mathcal{T}} \max\big(|\mathcal{U}_t|, 1\big)$, where a task with no distinct subtasks counts as a single estimator.

The multi-objective evaluation criterion Our goal is to evaluate the quality of each triplet $(x, c, m) \in \mathcal{X}\times\mathcal{C}\times\mathcal{M}$. To do this, we assess two competing quantities:

1. A **loss term** $\ell_{x,c,m}$, returned by the estimator m. This represents the degradation in downstream performance caused by the compression. A low loss indicates that the compression did not negatively impact the estimator's capabilities.
2. A **storage term** $s_{x,c}$, representing the final storage cost of the compressed signal z (e.g., file size). This term depends only on c and x, independent of m. A low value indicates that the compressor has significantly reduced the size of the data.

These two terms are not directly commensurable in their raw configuration. Firstly, they represent fundamentally different quantities. Secondly, the loss is not commensurable across models or datasets, since the underlying metric and its scale depend heavily on the specific task and data characteristics. Therefore, both terms are min–max normalised within each dataset x across the pool of compressors $\mathcal{C}$:

$$\tilde{\ell}_{x,c,m} = \frac{\ell_{x,c,m} - \min_{c'\in\mathcal{C}} \ell_{x,c',m}}{\max_{c'\in\mathcal{C}} \ell_{x,c',m} - \min_{c'\in\mathcal{C}} \ell_{x,c',m}}, \tilde{s}_{x,c} = \frac{s_{x,c} - \min_{c'\in\mathcal{C}} s_{x,c'}}{\max_{c'\in\mathcal{C}} s_{x,c'} - \min_{c'\in\mathcal{C}} s_{x,c'}} \tag{1}$$

We propose multi-objective selection criterion $L_{x,c,m}$, which is the convex combination of the two aforementioned normalised quantities:

$$L_{x,c,m}(\alpha) = (1-\alpha)\,\tilde{\ell}_{x,c,m} + \alpha\,\tilde{s}_{x,c} \quad \text{for } \alpha \in [0,1], \tag{2}$$

where α is the storage weight. A value of $\alpha = 0$ means that the selection is based only on the loss term, whereas $\alpha = 1$ values only the storage term. $\alpha = 0.5$ weights both terms equally. Algorithm 1 outlines the steps involved in applying

Algorithm 1 Two-phase evaluation pipeline.

Require: dataset $\mathcal{X}$, compressors $\mathcal{C}$, estimators $\mathcal{M}$, number of folds K
for each dataset $x \in \mathcal{X}$ **do**
  for $k \leftarrow 1$ **to** K **do**
    $x_{k,train}, x_{k,test} \leftarrow$ raw training and test set of fold k
  Phase 1 – fit
    fit every compressor $c \in \mathcal{C}$ and every estimator $m \in \mathcal{M}$ on $x_{k,train}$
  Phase 2 – compress and evaluate
    for each fitted compressor $c \in \mathcal{C}$ **do**
      $\hat{x_k} \leftarrow \textsc{Decompress}(\textsc{Compress}_c(x_{k,test}))$
      for each estimator $m \in \mathcal{M}$ **do**
        $\ell_{x,c,m,k} \leftarrow m(\hat{x_k}, x_{k,test})$
      end for
      record $s_{x,c,k}$
    end for
  Normalise across compressors
    $\tilde{\ell}_{x,c,m,k} \leftarrow \textsc{MinMax}_{c\in\mathcal{C}} \ell_{x,c,m,k}$
    $\tilde{s}_{x,c,k} \leftarrow \textsc{MinMax}_{c\in\mathcal{C}} s_{x,c,k}$
  end for
  Average over folds
  $\tilde{\ell}_{x,c,m} \leftarrow \text{mean}_k\ \tilde{\ell}_{x,c,m,k}$
  $\tilde{s}_{x,c} \leftarrow \text{mean}_k\ \tilde{s}_{x,c,k}$
  Multi-objective criterion
  for each $(c, m) \in \mathcal{C} \times \mathcal{M}$. **do**
    $L_{x,c,m} \leftarrow (1-\alpha)\,\tilde{\ell}_{x,c,m} + \alpha\,\tilde{s}_{x,c}$
  end for
end for
return $L_{x,c,m}, \tilde{\ell}_{x,c,m}, \tilde{s}_{x,c}$

the multi-objective criterion. The algorithm admits the possibility of following a K-fold cross-validation protocol. The number of folds K is a free parameter. At $K = 1$ it reduces to the case of no cross-validation.

The Price of Generality To ensure a fair evaluation of algorithms across different use cases, we propose two selection rules that differ in when the workload average is taken relative to the argmin.

The *task-specific* rule minimises the criterion per estimator m:

$$C^*_{\text{TS}}(x, m; \alpha) = \arg\min_{c\in\mathcal{C}} L_{x,c,m}(\alpha). \tag{3}$$

This returns the optimal compressor tailored specifically to model m at trade-off α.

Conversely, the *task-agnostic* rule averages over subtasks u, and then over tasks t, before selecting the compressor. In a scenario where certain tasks are more critical than others, this rule allows different weights μ_t to be assigned to each task (with $\mu_t \geq 0$ and $\sum_t \mu_t = 1$). The generic task-agnostic rule is:

$$C^*_{\text{TA}}(x; \boldsymbol{\mu}, \alpha) = \arg\min_{c\in\mathcal{C}} (1-\alpha) \sum_{t\in\mathcal{T}} \mu_t\, \mathbb{E}_{m\in\mathcal{M}_t}\left[\tilde{\ell}_{x,c,m}\right] + \alpha\,\tilde{s}_{x,c}. \tag{4}$$

Algorithm 2 Task-specific and task-agnostic selection, PoG calculation.

Require: dataset $\mathcal{X}$, compressors $\mathcal{C}$, estimators $\mathcal{M}$, $\tilde{\ell}_{x,c,m}$, $\tilde{s}_{x,c}$, α, $\boldsymbol{\mu}$
for each $x \in \mathcal{X}$ **do**
  Task-specific winner
  for each $m \in \mathcal{M}$ **do**
    $C^*_{\mathrm{TS}_\mathrm{x}}(m) \leftarrow \arg\min_{c \in \mathcal{C}} \; L_{x,c,m}$
  end for
  Task-agnostic winner
  $C^*_{\mathrm{TA}_\mathrm{x}} \leftarrow \arg\min_{c \in \mathcal{C}} (1-\alpha) \sum_t \mu_t \; \mathrm{mean}_{m \in \mathcal{M}_t} \; \tilde{\ell}_{x,c,t,u} + \alpha \, \tilde{s}_{x,c}$
  Price of generality
  for each $m \in \mathcal{M}$ **do**
    $\mathrm{PoG}_\mathrm{x}(m) \leftarrow L_{x,\, C^*_{\mathrm{TA}_\mathrm{x}},\, m} - L_{x,\, C^*_{\mathrm{TS}_\mathrm{x}}(m),\, m}$
  end for
end for
return $\mathbf{C}^*_{\mathrm{TS}}$, $\mathbf{C}^*_{\mathrm{TA}}$, **PoG** ▷ indexed by $x \in \mathcal{X}$

When all tasks are weighted equally ($\mu_t = 1/|\mathcal{T}|$) the outer sum becomes a uniform average over tasks. This yields the single best general-purpose compressor across all evaluators.

Finally, we define the *Price of Generality* (PoG) as the performance penalty incurred by deploying the globally optimal compressor instead of the tailor-made one for a specific model m:

$$\mathrm{PoG}(x, m; \boldsymbol{\mu}, \alpha) = L_{x,\, C^*_{\mathrm{TA}}(x;\boldsymbol{\mu},\alpha),\, m}(\alpha) - L_{x,\, C^*_{\mathrm{TS}}(x,m;\alpha),\, m}(\alpha) \;\geq\; 0. \tag{5}$$

By definitions in Eq. 3 and Eq. 4, PoG is strictly non-negative. Algorithm 2 shows the application of the two selection rules and the PoG calculation.

The decision boundary visualisation Geometrically, each compressor acts as a point in the loss–storage plane, and sweeping α across $[0, 1]$ traces the lower-left convex hull of the Pareto front between these two competing objectives.

Extending this selection across the expected workload mix $\boldsymbol{\mu}$ provides a much broader decision framework. For a workload of $n_t = |\mathcal{T}|$ tasks, the set of all possible task weightings $\boldsymbol{\mu}$ forms an $(n_t - 1)$-dimensional simplex $\Delta^{n_t - 1}$. Because the task-agnostic loss (Eq. 4) is linear with respect to $\boldsymbol{\mu}$, for any fixed storage preference α, the simplex is partitioned into a set of convex polytopes. Inside each bounding region, a single compressor strictly minimises the global loss.

Each point of the simplex is a possible workload mix $\boldsymbol{\mu}$ and it is coloured by the compressor that Eq. 4 selects there. An example using $n_t = 3$ is shown in Figure 1. Once α and the expected task distribution $\boldsymbol{\mu}$ are defined, the optimal compressor can be identified directly from the map.

4 Experiments

In Section 4.1 we present the used dataset, the chosen compression methods and the evaluators of the workload. In Section 4.2 we introduce the metrics through which task quality is scored. Section 4.3 reports the results and Section 4.4 discusses their implications and limitations.

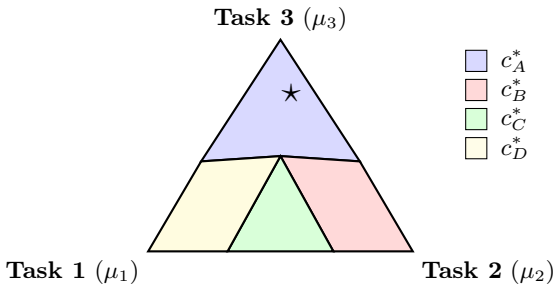


Fig. 1. Schematic of the workload simplex (Δ^2) for $n = 3$ tasks at a fixed storage preference α. The task-agnostic objective is linear in the task weights $\boldsymbol{\mu}$, so the space partitions into convex regions, each minimised by one compressor c^* (legend). $\star$ marks one possible real-world mix, in which Task 3 is more probable then the other two.

4.1 Experimental Settings

To ensure that our dataset pool $\mathcal{X}$ reflects the diversity of real-world digital twin deployments, we select five datasets spanning distinct operational domains. UCI-HAR and OPPORTUNITY capture high-frequency wearable kinematics for operator tracking; Hydraulic provides industrial condition monitoring from a physical test rig; and Beijing and Metro represent low-frequency smart-city infrastructure (environmental and traffic monitoring, respectively). These selections provide a robust mix of channel counts, temporal dynamics, and domain characteristics suitable for our mixed workload. Missing values are forward- then back-filled, ensuring every series is gap-free. The technical characteristics of the pool are summarized in Table 1a.

The compression methods used in our experiments are listed in Table 1b. Each compressor family is instantiated at the target compression ratios $2, 3, 4, 6, 8$. The compression ratio CR = bytes(original)/bytes(stored) is a quantity we use for reporting only as it does not depend on the initial size of the datasets, so its easier to read. The pool spans multiple paradigms and differs significantly in fitting cost. Downsampling, LTTB, DCT, and DWT are fixed transforms and require no fitting. Sim-Piece is natively parameterized by an error bound ε, thus we perform an ε-search on the training window to match the target compression ratio grid of the other families. Principal Component Analysis (PCA) requires a single $v \times v$ eigendecomposition (where v is the number of dataset channels), whereas the Autoencoder (AE) is the only method requiring iterative training via gradient descent. Furthermore, we distinguish between channel-dependent (CD) families (PCA, AE), which mix information across channels, and channel-independent (CI) families (the remainder), which compress each channel in isolation.

After being compressed and reconstructed, the data serves a mixed workload $\mathcal{T}$ of three distinct tasks: forecasting, classification, and aggregate querying. We chose these families because they reflect how a stored digital-twin history is consumed in practice. Forecasting anticipates the asset's future state (e.g., predictive maintenance or control); classification recognizes its operating mode or

(a) Dataset portfolio ($\mathcal{X}$): domains and technical dimensions.

Dataset	Abbr.	Domain	Rate [Hz]	Ch.	Rows	Cls.	Class. label
UCI-HAR [2]	UC	wearable	50	9	∼659k	6	activity
OPPORTUNITY [24]	OP	wearable	30	9	∼51k	5	activity
Hydraulic [12]	HY	industrial	1	8	∼132k	4	valve cond.
Beijing [31]	BE	environmental	1/3600	11	∼35k	4	wind quadrant
Metro [13]	ME	mobility	1/3600	5	∼41k	2	weekday/weekend

(b) Summary of the compressor pool ($\mathcal{C}$): approach description and corresponding reconstruction method.

Family	Approach	Reconstruction	Chan.	Training cost
Downsample	temporal decimation	linear interpolation	CI	none
LTTB [25]	perceptual decimation	linear interpolation	CI	none
DCT [34]	spectral (cosine)	inverse DCT	CI	none
DWT [7]	spectral (wavelet)	inverse DWT	CI	none
Sim-Piece [17]	piecewise-linear model	evaluate segments	CI	ε-search
PCA [26]	linear projection	basis reprojection	CD	eigendecomposition
Autoencoder [3]	learned (neural)	decoder network	CD	iterative training

(c) Workload and associated references. For forecasting, h represents the evaluated horizons (the negative values account for its backward-looking facet).

Task (t)	**Subtask** (u)	**Model/Estimator** (m)	**Ref.**
Forecasting	$h \in \{\pm 1, \pm 12, \pm 36\}$	OLS with time-delay embedding	[1]
Classification	None	DTW-1NN	[9]
Aggregate-query	`mean`, `max`, `min`, `std`	Bucket-level operators (width 64)	–

condition (e.g., activity or fault detection); and aggregate queries represent the everyday analytical workload of any TSDB (e.g., dashboards, alerts, and windowed reports).

Table 1c outlines the specific subtasks $\mathcal{U}_t$ and estimators $\mathcal{M}$ assigned to each task t. For forecasting, we use the OLS (Ordinary Least Squares) model, which fits a closed-form linear map from a fixed window of recent samples to the value h steps ahead. The values of h are forecasting's subtasks and they include $h \in \{1, 12, 36\}$ for forward prediction at horizons as well as a $h \in \{-1, -12, -36\}$ for backward generalization (*past* subtask). For classification, we use the DTW-1NN (Dynamic Time Warping-1-Nearest-Neighbor), the canonical time-series-classification baseline, which labels a window by its nearest training exemplar under dynamic time warping. The aggregate operators are standard ($\{\texttt{mean}, \texttt{max}, \texttt{min}, \texttt{std}\}$) and the bucket used to compute them has width 64.

To isolate the impact of compression on the downstream tasks, we intentionally utilize minimal, deterministic estimators (OLS and DTW-1NN). Neither has meaningful hyperparameters or training randomness, so any change in task performance is attributable to the compression rather than to model capacity or tuning.

By keeping the models simple and shared across all compressors, we ensure that changes in task quality are strictly attributable to the fidelity of the reconstructed signal $\hat{x}$, rather than complex model dynamics or stochastic training variance. The forecasting and classification estimators are trained on the raw data, but inference is performed on the reconstructed signal $\hat{x}$. The aggregates are computed on raw windows and compared directly against aggregates computed on $\hat{x}$. It is worth noting that the compressors' absolute scores are conditional on this model choice; adopting more complex estimators might rescale the losses and, in some edge cases, reorder the optimal compressors.

4.2 Evaluation

This section instantiates the loss term $\ell_{x,c,m}$ returned by each estimator and the storage term $s_{x,c}$ of the multi-objective criterion (Eq. 2).

For the loss term, we use NMSE (Normalised Mean Squared Error) for forecasting and aggregation and Macro-F_1 for classification. NMSE is used for forecasting and aggregation. NMSE measures how far the result lands from the truth: NMSE=0 means a perfect prediction and above NMSE>1 means worse than a naive prediction using the average. Macro-F_1 computes the F_1 score separately for each class and then averages them with equal weight. In this way, a model cannot look good by getting only the frequent classes right. We use $1-\text{macro-}F_1$ as the loss for classification, so that lower is better for both metrics.

For the storage term s, we count the *bytes* each compressor retains, taken before any byte-encoding back end, the final lossless layer that squeezes those values into their on-disk form [4]. Counting before this layer makes the comparison rank the compressors, not the encoders that pack their output. The storage term excludes model parameters (autoencoder weights, PCA bases) and depends only on the compressor, never on the task. It is an absolute *cost* (fewer bytes is better), and this cost is what becomes $\tilde{s}_{x,c}$ in Equation 2.

Every dataset is split into $K = 5$ folds following a purged, embargoed, block-disjoint cross-validation [23], which prevents train–test data leakage.

The headline setting is $\alpha = 0.5$ and $\boldsymbol{\mu}$ uniform.

4.3 Results

Table 2 shows per-subtask winning compressor $C^*_{\text{TS}}(x; m)$ (Eq. 3) and the task-agnostic pick $C^*_{\text{TA}}(x)$ (Eq. 4). Each cell reports the winning `family` and its target compression ratio `N`. Past appears as a single line because its results are averaged over its horizons ($\{-1, -12, -36\}$).

The table shows that two to five distinct compressors win on every dataset, and none wins everywhere. Holding the dataset fixed, the winner depends on what the task needs. The windowed `mean` is won by the smooth low-frequency `dct_cr8` on three of the five datasets. The extrema queries (`max`, `min`, `std`) often diverge from the `mean` within a dataset, e.g. on Hydraulic the `mean` is won by `sim_piece` but the `min`/`max` by `dwt`. On three over five dataset the forecasting

moves with the horizon, e.g. on Metro the winner moves from `sim_piece` at short horizons to the smooth `dct` at long ones. Holding the task fixed, the winner still shifts from stream to stream as the channel structure changes. Classification shows this sharply. The two channel-dependent methods, PCA and AE, win where cross-channel structure is exploitable (PCA on Hydraulic, the autoencoder on UCI-HAR), while the channel-independent win on the rest. Also forecasting and aggregate winners change depending on the dataset.

Table 2. Per-subtask winning compressor and per-dataset task-agnostic pick. Each cell reports the winning `compressor` and its target CR `N`. Columns are datasets, keyed by the *Abbr.* of Table 1a.

subtask	BE	HY	ME	OP	UC
past	`dct_8`	`dct_8`	`sim_piece_8`	`dwt_8`	`dct_8`
h=1	`dct_8`	`dct_8`	`sim_piece_8`	`dwt_8`	`ae_8`
h=12	`downsample_8`	`dct_8`	`dct_8`	`dwt_8`	`dct_8`
h=36	`downsample_8`	`dct_8`	`dct_8`	`dct_8`	`dct_8`
classification	`dct_8`	`pca_6`	`sim_piece_8`	`sim_piece_8`	`ae_8`
mean	`dct_8`	`sim_piece_8`	`sim_piece_8`	`dct_8`	`dct_8`
min	`downsample_8`	`dwt_8`	`sim_piece_8`	`lttb_8`	`ae_8`
max	`sim_piece_8`	`dwt_8`	`sim_piece_8`	`downsample_8`	`ae_8`
std	`dct_8`	`dct_8`	`sim_piece_8`	`lttb_8`	`ae_8`
C^*_{TA}	`dct_8`	`sim_piece_8`	`sim_piece_8`	`lttb_8`	`ae_8`

Table 3a shows the *Price of Generality* (Eq. (5)) per (dataset, subtask), scaled so 0% is the task's best compressor and 100% the worst.

Table 3a shows the *Price of Generality* per (dataset, subtask): each entry is the value of Eq. (5) multiplied by 100, i.e. the excess normalised loss of the task-agnostic pick over the subtask's best compressor, in percentage points. The values are mostly small: most of them are below 5% or exactly zero where the task-agnostic pick is equal to the task-specific one. The largest is 15.6% for classification on Hydaulic [12].

Table 3b reports the worst-matched task-specific compressor for every dataset: its excess workload-average loss over $C^*_{\mathrm{TA}}(x)$ ranges from +49% (on UCI-HAR) to 401% (on Hydraulic). The excess is relative to each $C^*_{\mathrm{TA}}(x)$ loss, so where that base is small the percentage inflates. Metro is such a case: the base is only 0.014, so its h=12 specialist reads +1166%, yet in absolute terms its excess (0.16) is below Hydraulic's (0.26), which remains the largest real mis-specialisation.

Finally, figure 2 shows the bounding regions for OPPORTUNITY: the winning compressor over the workload simplex (vertices: the three task types) at three values of α. Sweeping α traces the convex hull of the pool's Pareto front: every winner $C^*_{\mathrm{TA}}(x; \alpha, \boldsymbol{\mu})$ the algorithm returns is a supported Pareto-optimal compressor for that specific wokload mix $\boldsymbol{\mu}$.

(a) Price of generality (PoG) per (dataset, subtask). Each entry is the value of Eq. (5) multiplied by 100. Because the objective is min–max normalised, this is the excess normalised loss of the task-agnostic pick over the subtask's best compressor, in percentage points. 0 means C^*_{TA} is itself the subtask winner. Datasets are keyed by the *Abbr.* of Table 1a.

subtask	BE	HY	ME	OP	UC
past	0.4	0.2	0.0	2.9	2.6
$h = 1$	0.0	0.1	0.0	2.8	0.0
$h = 12$	3.6	0.3	3.6	3.2	7.5
$h = 36$	0.8	0.1	3.5	5.2	9.5
classification	0.0	**15.6**	0.0	1.9	0.0

subtask	BE	HY	ME	OP	UC
mean	0.0	0.0	0.0	2.6	3.9
min	2.7	0.5	0.0	0.0	0.0
max	6.9	0.3	0.0	0.1	0.0
std	0.0	9.7	0.0	0.0	0.0

(b) Cost of mis-specialisation per dataset: the worst-matched task-specific compressor and its excess workload-average loss over $C^*_{\mathrm{TA}}(x)$, computed using normalised-objective units (≤ 1).

Dataset	C^*_{TA}	worst subtask specialist	ΔL	excess
HY	`sim_piece_8`	`pca_6` (classif.)	0.26	+401%
ME	`sim_piece_8`	`dct_8` (h=12)	0.16	+1166%
BE	`dct_8`	`sim_piece_8` (agg:max)	0.09	+88%
OP	`lttb_8`	`sim_piece_8` (classif.)	0.08	+47%
UC	`ae_8`	`dct_8` (h=12)	0.04	+49%

4.4 Discussion

The results show that no compressor dominates the full portfolio, so it is not optimal to pick one blindly. Morover, committing to the task-agnostic winner costs at most 15.6% of the achievable loss on any task, whereas committing to the wrong task-specialist inflates the workload-average loss much more: the largest absolute gap is $\Delta L = 0.26$ on Hydraulic (+401%), while the largest relative increase is +1166% on Metro, where the loss base is small. The task-agnostic pick is therefore a favourable hedge: a bounded downside against the far larger cost of over-committing to one task's specialist.

One caveat is that forecasting is not meaningful everywhere: on Hydraulic and Metro the forecasting model does no better than predicting the average (NMSE ≥ 1), so the forecasting gap there compares two equally useless predictors.

A further consideration concerns the estimators. We deliberately fix them minimal and deterministic. This means a compressor is scored on what a simple estimator can recover, not on the task-relevant information it preserves in principle. It could retain structure that only a stronger, non-linear estimator can exploit so under our protocol it may appear worse than it is. A legitimate alternative is to score each compressor by the performance of the *best* estimator for the task and the dataset. We leave this comparison under task-optimal estimators to future work.

Table 2 also reveals which compressors are effective: parameter-free methods outperform others on the vast majority of the (datset, subtask) combinations. The deployed C^*_{TA} is a learned autoencoder on only one dataset (UCI-HAR). In the multi-objective criterion, we chose not to take into account the parameter

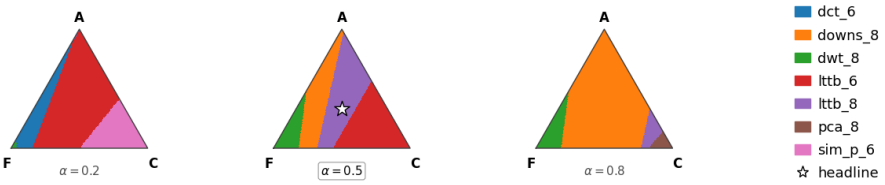


Fig. 2. Winning compressor over the workload simplex for [24]. Each triangle corresponds to a different value of α. The vertices represent forecasting (forward+past) (**F**), classification (**C**) and aggregate-query (**A**). The triangle are coloured by the winner compressor given with Eq. 4. Each colour is a compressor labelled `family_CR`.

space required by the learned methods (autoencoder weights, PCA bases). If we did so, the role of the autoencoder would be even more marginal. This means that, in general, cheaper methods offer the same performance at the same loss-storage trade-off.

When the workload mix can be anticipated, the $(\alpha, \boldsymbol{\mu})$ sweep (Fig. 2) turns the study into a lookup table. We believe that the value of the study does not reside in the particular winners on our five datasets but in the decision procedure behind them. This is what makes an *offline* study worthwhile: selecting a compressor is a design-time decision, so the analysis is run once on representative history and then amortised. The downside is that an offline pick is static. If the stream drifts, the chosen compressor can go stale, which is precisely why an online, adaptive selection is meaningful.

5 Conclusion

This paper demonstrates that the optimal compressor varies depending on the task and dataset: no single compressor consistently outperforms the others, and the variation is a consequence of the signal's characteristics that each compressor preserves. Moreover, making a single task-agnostic commitment is inexpensive: the cost of it is at most 15.6% of the per-cell loss range. By contrast, committing to a task-specific compressor tuned for the wrong task can inflate the workload loss sharply: the largest *absolute* gap is $\Delta L = 0.26$ (Hydraulic, +401%), while the largest *relative* increase is +1166% (Metro, where the loss base is small). This makes the task-agnostic compressor a safe default whenever the workload is heterogeneous or not known in advance. When the task mix can be estimated, the $(\alpha, \boldsymbol{\mu})$ policy sweep refines this choice: it exposes how the optimal compressor shifts as the storage weight α and the per-task weights $\boldsymbol{\mu}$ vary, letting a practitioner choose a compressor better matched to the expected workload.

From this work, two directions follow. First, the offline pick could be refreshed *online*: a drift detector would trigger re-selection when the signal or workload shifts, turning this offline study into a continuously-updated evaluation driven by the task-aware multi-objective criterion proposed here. Second, the evaluation could be re-run with stronger, task-optimal estimators (e.g. deep forecasters or modern time-series classifiers) to test how robust the winner and price-of-generality conclusions are to estimator capacity.

Disclosure of Interests. The authors have no competing interests to declare.

Generative AI use Anthropic's Claude was used to assist with drafting and editing prose, tables and figures. All outputs were reviewed and verified by the authors.

References

1. Acito, F.: Ordinary least squares regression. In: Predictive Analytics with KNIME: Analytics for Citizen Data Scientists, pp. 105–124. Springer Nature Switzerland (2023). https://doi.org/10.1007/978-3-031-45630-5_6, https://doi.org/10.1007/978-3-031-45630-5_6
2. Anguita, D., Ghio, A., Oneto, L., Parra, X., Reyes-Ortiz, J.L.: A Public Domain Dataset for Human Activity Recognition Using Smartphones. Computational Intelligence (2013)
3. Bank, D., Koenigstein, N., Giryes, R.: Autoencoders (Apr 2021). https://doi.org/10.48550/arXiv.2003.05991
4. Chiarot, G., Silvestri, C.: Time Series Compression Survey. ACM Computing Surveys **55**(10), 1–32 (Oct 2023). https://doi.org/10.1145/3560814
5. Chowdhury, M.R., Tripathi, S., De, S.: Adaptive Multivariate Data Compression in Smart Metering Internet of Things. IEEE Transactions on Industrial Informatics **17**(2), 1287–1297 (Feb 2021). https://doi.org/10.1109/TII.2020.2981382
6. Cover, T.M., Thomas, J.A.: Elements of Information Theory. Wiley, 2nd edn. (2006). https://doi.org/10.1002/047174882X
7. Daubechies, I.: Orthonormal bases of compactly supported wavelets. Communications on Pure and Applied Mathematics **41**(7), 909–996 (Oct 1988). https://doi.org/10.1002/cpa.3160410705
8. Emmerich, M.T.M., Deutz, A.H.: A tutorial on multiobjective optimization: Fundamentals and evolutionary methods. Natural Computing **17**(3), 585–609 (Sep 2018). https://doi.org/10.1007/s11047-018-9685-y
9. Faouzi, J.: Time Series Classification: A Review of Algorithms and Implementations. In: Rocha, J., M. Viana, C., Oliveira, S. (eds.) Time Series Analysis - Recent Advances, New Perspectives and Applications. IntechOpen (2024). https://doi.org/10.5772/intechopen.1004810, https://www.intechopen.com/chapters/1185930
10. Grieves, M., Vickers, J.: Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems. In: Transdisciplinary Perspectives on Complex Systems, pp. 85–113. Springer, Cham (2017)
11. Han, L., Ye, H.J., Zhan, D.C.: The Capacity and Robustness Trade-Off: Revisiting the Channel Independent Strategy for Multivariate Time Series Forecasting. IEEE Transactions on Knowledge and Data Engineering **36**(11), 7129–7142 (2024). https://doi.org/10.1109/TKDE.2024.3400008, https://ieeexplore.ieee.org/document/10529618/
12. Helwig, N., Pignanelli, E., Schütze, A.: Condition monitoring of a complex hydraulic system using multivariate statistics. In: 2015 IEEE International Instrumentation and Measurement Technology Conference (I2MTC). pp. 210–215. IEEE, Pisa, Italy (2015). https://doi.org/10.1109/I2MTC.2015.7151267, uCI Machine Learning Repository, dataset 447: Condition monitoring of hydraulic systems

13. Hogue, J.: Metro Interstate Traffic Volume. UCI Machine Learning Repository, dataset 492 (2019). https://doi.org/10.24432/C5X60B, https://archive.ics.uci.edu/dataset/492, no associated publication; dataset donated to the UCI repository. Hourly Minneapolis–St. Paul, MN traffic volume for westbound I-94 with weather features
14. Jensen, S.K., Pedersen, T.B., Thomsen, C.: Time Series Management Systems: A Survey. IEEE Transactions on Knowledge and Data Engineering **29**(11), 2581–2600 (Nov 2017). https://doi.org/10.1109/TKDE.2017.2740932
15. Jensen, S.K., Thomsen, C., Pedersen, T.B.: ModelarDB: Integrated Model-Based Management of Time Series from Edge to Cloud. In: Hameurlain, A., Tjoa, A.M. (eds.) Transactions on Large-Scale Data- and Knowledge-Centered Systems LIII, vol. 13840, pp. 1–33. Springer Berlin Heidelberg, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-662-66863-4_1
16. Khattach, O., Moussaoui, O., Hassine, M.: End-to-End Architecture for Real-Time IoT Analytics and Predictive Maintenance Using Stream Processing and ML Pipelines. Sensors **25**(9), 2945 (Jan 2025). https://doi.org/10.3390/s25092945
17. Kitsios, X., Liakos, P., Papakonstantinopoulou, K., Kotidis, Y.: Sim-Piece: Highly Accurate Piecewise Linear Approximation through Similar Segment Merging. Proceedings of the VLDB Endowment **16**(8), 1910–1922 (Apr 2023). https://doi.org/10.14778/3594512.3594521
18. Landau, O., Nissim, N.: Improving Generalization Capabilities of Models Trained on Time Series Data Using Novel Downsampling Methods. In: 2025 IEEE International Conference on Data Mining (ICDM). pp. 407–416. IEEE, Washington DC, DC, USA (Nov 2025). https://doi.org/10.1109/ICDM65498.2025.00048
19. Liu, Z., Zeng, P., Ding, Y.: Predictability-Aware Compression and Decompression Framework for Multichannel Time Series Data with Latent Seasonality. In: Proceedings of the ACM Web Conference 2026. pp. 5525–5536 (Apr 2026). https://doi.org/10.1145/3774904.3792488
20. Miao, Z., Li, W., Pan, X.: Multivariate time series collaborative compression for monitoring systems in securing cloud-based digital twin. Journal of Cloud Computing **13**(1), 16 (Jan 2024). https://doi.org/10.1186/s13677-023-00579-4
21. Muñiz-Cuza, C.E., Boehm, M., Pedersen, T.B.: CAMEO: Autocorrelation-Preserving Line Simplification for Lossy Time Series Compression (Jan 2025). https://doi.org/10.48550/arXiv.2501.14432
22. Muniz-Cuza, C., Jensen, S., Brusokas, J., Ho, N., Pedersen, T.: Evaluating the impact of error-bounded lossy compression on time series forecasting. In: Advances in Database Technology - EDBT. pp. 650–663. No. 3 in Advances in Database Technology - EDBT, OpenProceedings (Mar 2024). https://doi.org/10.48786/edbt.2024.56, publisher Copyright: © 2024 Copyright held by the owner/author(s).; 27th International Conference on Extending Database Technology, EDBT 2024 ; Conference date: 25-03-2024 Through 28-03-2024
23. Lopez de Prado, M.: Advances in Financial Machine Learning. John Wiley & Sons, Hoboken, NJ (2018)
24. Roggen, D., Calatroni, A., Rossi, M., Holleczek, T., Förster, K., Tröster, G., Lukowicz, P., Bannach, D., Pirkl, G., Ferscha, A., Doppler, J., Holzmann, C., Kurz, M., Holl, G., Chavarriaga, R., Sagha, H., Bayati, H., Creatura, M., Millàn, J.d.R.: Collecting complex activity datasets in highly rich networked sensor environments. In: 2010 Seventh International Conference on Networked Sensing Systems (INSS). pp. 233–240 (Jun 2010). https://doi.org/10.1109/INSS.2010.5573462

25. Rui, L., Huang, X., Song, S., Wang, C., Wang, J., Cao, Z.: Largest triangle sampling for visualizing time series in database. Proceedings of the ACM on Management of Data **3**(1), 1–26 (2025)
26. Shlens, J.: A Tutorial on Principal Component Analysis (Apr 2014). https://doi.org/10.48550/arXiv.1404.1100
27. Signoretti, G., Silva, M., Andrade, P., Silva, I., Sisinni, E., Ferrari, P.: An Evolving TinyML Compression Algorithm for IoT Environments Based on Data Eccentricity. Sensors **21**(12), 4153 (Jun 2021). https://doi.org/10.3390/s21124153
28. Struckov, A., Yufa, S., Visheratin, A.A., Nasonov, D.: Evaluation of modern tools and techniques for storing time-series data. Procedia Computer Science **156**, 19–28 (2019). https://doi.org/10.1016/j.procs.2019.08.125
29. Underwood, R., Calhoun, J.C., Di, S., Cappello, F.: Understanding The Effectiveness of Lossy Compression in Machine Learning Training Sets (Mar 2024). https://doi.org/10.48550/arXiv.2403.15953
30. Wingerath, W., Gessert, F., Friedrich, S., Ritter, N.: Real-time stream processing for Big Data. it – Information Technology **58**(4), 186–194 (2016). https://doi.org/10.1515/itit-2016-0002
31. Zhang, S., Guo, B., Dong, A., He, J., Xu, Z., Chen, S.X.: Cautionary tales on air-quality improvement in Beijing. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences **473**(2205), 20170457 (2017). https://doi.org/10.1098/rspa.2017.0457, uCI Machine Learning Repository, dataset 501: Beijing Multi-Site Air-Quality Data
32. Zhang, Y., Zhang, F., Li, H., Zhang, S., Guo, X., Chen, Y., Pan, A., Du, X.: Data-Aware Adaptive Compression for Stream Processing. IEEE Transactions on Knowledge and Data Engineering **36**(9), 4531–4549 (2024). https://doi.org/10.1109/TKDE.2024.3377710, https://ieeexplore.ieee.org/document/10473167/
33. Zhao, L., Shen, Y.: Rethinking channel dependence for multivariate time series forecasting: Learning from leading indicators. In: International Conference on Learning Representations. vol. 2024, pp. 41433–41451 (2024)
34. Zhou, J.: On discrete cosine transform (Sep 2011). https://doi.org/10.48550/arXiv.1109.0337
35. Zou, X., Wang, S., Shi, Y., Chen, X., Jin, S., Tao, D., Xia, W.: The Design of an Efficient Lossy Compressor for Time Series Databases. ACM Trans. Archit. Code Optim. **22**(4), 131:1–131:27 (2025-12-13). https://doi.org/10.1145/3767158, https://dl.acm.org/doi/10.1145/3767158